MATERIAL DE AULA - SEMANA 01 - REACT

Introdução ao React e configuração do ambiente

Unidade Curricular: Desenvolvimento de Frameworks - React
Projeto em sala: To-Do Pro - Gestão de tarefas fullstack
Carga sugerida: 3h45min
Nível: Introdução com aplicação orientada à prática
Perfil da aula: condução em sala com explicação, demonstração e execução guiada

---

1. Objetivo da aula

O objetivo desta aula é apresentar o React como uma ferramenta para desenvolvimento de interfaces web modernas, mostrar como ele se conecta ao contexto do frontend e do fullstack, preparar o ambiente de desenvolvimento e iniciar a construção de uma primeira versão do projeto To-Do Pro.

Ao final da aula, os alunos deverão ser capazes de:

- compreender a diferença entre frontend, backend e fullstack;
- entender o papel do React no desenvolvimento de interfaces;
- verificar a instalação de Node.js, npm e Git;
- criar um projeto com Vite;
- entender a estrutura inicial de um projeto React;
- escrever um primeiro componente com JSX;
- iniciar a lógica de estado local com `useState`;
- organizar o código em componentes simples;
- registrar o progresso no Git e documentar a execução do projeto.

---

2. Contextualização da aula

Uma empresa ainda organiza tarefas em planilhas, mensagens e listas dispersas. Isso gera duplicidade, erros, baixa visibilidade e dificuldade para acompanhar o que foi concluído.

O To-Do Pro, uma aplicação para cadastrar, acompanhar e concluir tarefas. A ideia é que a turma, ao longo da semana, construa uma solução completa com foco em experiência do usuário, organização do código, manipulação de dados e evolução estrutural da aplicação.

Nesta primeira semana, o foco não é a solução final completa, e sim a base técnica necessária para que os alunos entendam o funcionamento do React e consigam transformar a ideia em uma interface funcional.

---

3. Metodologia sugerida para a aula

A aula pode ser conduzida em quatro blocos:

1. Aula expositiva com exemplos e analogias.
2. Demonstração prática no terminal e no navegador.
3. Execução orientada pelos alunos.
4. Sistematização com discussão, validação e organização do código.

É importante que o professor mantenha uma sequência clara:

- primeiro explicar;
- depois demonstrar;
- depois fazer junto com a turma;
- por fim, incentivar a exploração individual ou em duplas.

---

4. Bloco 1 — Conceitos fundamentais

4.1 Frontend, backend e fullstack

Comece perguntando aos alunos:

- O que é a parte do sistema que o usuário vê?
- O que acontece quando um botão envia dados para o servidor?
- Onde as tarefas ficam guardadas?

A partir disso, explique a arquitetura da aplicação:

```
None
Usuário -> Interface React -> API do backend -> Banco de dados
```

```
Usuário <- Interface React <- Resposta da API <- Banco de dados
```

Explicação

- Frontend é a camada visual e interativa.
- Backend é a parte que processa regras e validações.
- Banco de dados guarda as informações de forma persistente.
- Fullstack é a integração dessas camadas.

Use exemplos simples, como um formulário de tarefa:

- o frontend mostra o campo de texto;
- o backend valida se o título foi preenchido;
- o banco salva a tarefa;
- o frontend atualiza a tela com o novo item.

4.2 O que é React?

O React é uma biblioteca JavaScript para construção de interfaces de usuário. Ele surgiu em 2013, no Facebook, a partir da necessidade de criar interfaces mais rápidas, organizadas e fáceis de manter em aplicações complexas, como o próprio feed de notícias da rede social.

Na época, o problema era comum: muitos sites e aplicações web estavam ficando pesados, com muito código JavaScript manipulando o DOM diretamente. Isso tornava a manutenção difícil, especialmente quando a interface precisava responder a muitos eventos e atualizar partes específicas da tela sem recarregar a página inteira.

O React trouxe uma ideia inovadora: em vez de manipular o DOM de forma imperativa e manual, o desenvolvedor descreve a interface em função do estado atual do sistema. Quando os dados mudam, o React recalcula a interface e atualiza apenas o que foi necessário.

Essa abordagem foi uma grande mudança no desenvolvimento frontend, porque a lógica de interface passou a ser pensada em termos de componentes, estados e renderização declarativa, em vez de comandos diretos sobre elementos da página.

Com o tempo, o React evoluiu bastante. Ele passou de uma biblioteca focada em componentes funcionais simples para um ecossistema completo, com conceitos como:

- componentes reutilizáveis;
- estados e props;
- hooks, como `useState`, `useEffect` e `useMemo`;
- renderização em servidor e otimização de desempenho;
- suporte a aplicações grandes, com arquitetura modular e escalável;
- integração com bibliotecas de roteamento, gerenciamento de estado e UI.

Essa evolução mostrou que o React não foi apenas uma ferramenta pontual, mas uma mudança de paradigma para o desenvolvimento frontend moderno.

Exemplo de componentes da aplicação:

```
None
App
├── Header
├── TaskForm
├── TaskList
│   └── TaskItem
└── TaskFilters
```

O React funciona com abordagem declarativa: o desenvolvedor descreve como a tela deve ficar para determinado estado, e a biblioteca atualiza a interface quando os dados mudam.

4.3 React e Angular: pontos de contato e diferenças

Como vocês já fizeram Angular, esta é uma oportunidade excelente de usar o conhecimento anterior como ponte. A comparação ajuda a reduzir a resistência inicial ao React.

Quadro comparativo com foco no que muda:

Conceito	Angular	React
Natureza	Framework completo	Biblioteca de UI
Estrutura	Componentes com template, classe e metadados	Componentes em função com JSX
Template	HTML + diretivas	JSX mistura JavaScript e marcação
Estado	Propriedade da classe/sinais	`useState` e outros hooks
Atualização	Detecção de alterações do framework	Re-renderização declarativa

4.4 DOM, árvore de elementos e renderização

Exemplo de DOM Tradicional

HTML:

```
HTML
<main>
  <h1>To-Do Pro</h1>
```

```
  <button>Nova tarefa</button>
</main>
```

Árvore do DOM

```
None
document
└── main
    ├── h1
    │   └── "To-Do Pro"
    └── button
        └── "Nova tarefa"
```

JavaScript imperativo: Alterar o Texto do “h1”

```
None
const title = document.querySelector("h1");
title.textContent = "Tarefas de hoje";
```

Esse código funciona, mas exige que o programador saiba exatamente quais elementos existem, quando devem ser alterados e como isso afeta a interface.

A abordagem declarativa do React:

```
None
function TaskTitle({ title, highlighted }) {
  return (
    <h1 className={highlighted ? "highlight" : ""}>
      {title}
    </h1>
  );
}
```

Nessa abordagem o código descreve o resultado desejado, e o React decide como atualizar o DOM real.

4.5 Virtual DOM e reconciliação

Entenda de forma simples:

1. algo no state muda;
2. o React executa o componente novamente;
3. retorna uma nova árvore de elementos;
4. compara com a anterior;
5. atualiza somente o necessário no DOM real.

o React não “refaz a página inteira”; ele identifica a demanda e altera somente o componente

4.6 A importância das keys

Quando a interface renderiza listas, React precisa identificar cada item de forma estável.

```
None
{tasks.map((task) => (
  <li key={task.id}>{task.title}</li>
))}
```

A `key` deve ser única e estável. Quando ela é baseada em índice, a lista pode apresentar comportamento confuso após remoções ou reordenação.

---

5. Bloco 2 — Preparando o ambiente

5.1 VS Code

Explique que o VS Code será o ambiente principal de desenvolvimento. Recomendado mostrar a interface básica:

- explorer;
- terminal integrado;
- editor;
- extension marketplace.

Sugestão de extensões:

- ESLint
- Prettier
- Error Lens
- JavaScript and TypeScript Nightly

Explique que essas extensões ajudam, mas não substituem a leitura das mensagens de erro.

5.2 Node.js e npm

Comandos no terminal:

```
None
node --version
npm --version
```

O que cada um representa:

- `node` permite executar JavaScript fora do navegador;
- `npm` gerencia pacotes e scripts do projeto.

Atividade de sala: checklist do ambiente

Criar um arquivo chamado `checklist-ambiente.md` com itens como:

```
None
# Checklist do ambiente

- [ ] VS Code instalado
- [ ] Node.js funcionando
- [ ] npm funcionando
- [ ] Git funcionando
- [ ] Navegador atualizado
- [ ] Terminal integrado funcionando
```

---

6. Bloco 3 — Criando o projeto com Vite

6.1 Introdução ao Vite

O Vite é uma ferramenta moderna para iniciar, desenvolver e compilar projetos frontend, incluindo aplicações React. Ele foi criado para resolver um problema muito comum no frontend: a necessidade de um ambiente de desenvolvimento mais rápido, leve e eficiente, especialmente em projetos que crescem e passam a ter muitas dependências.

No começo, ferramentas como Webpack e Create React App eram bastante usadas para configurar projetos. Embora funcionassem, elas frequentemente tinham tempos de inicialização mais lentos, build mais pesado e estrutura mais complexa para projetos em crescimento. O Vite apareceu como uma alternativa mais moderna, usando uma abordagem baseada em módulos ES e carregamento sob demanda.

A ideia central do Vite é simples: em vez de empacotar tudo no início, ele serve o código do projeto de forma mais direta e só compila os módulos necessários quando são acessados. Isso torna o desenvolvimento muito mais ágil, com reinicialização quase imediata e melhor experiência para quem está escrevendo código.

Além disso, o Vite é usado para criar projetos React, Vue e outros frameworks, mas o importante aqui é entender que ele é uma ferramenta de configuração e desenvolvimento, não o próprio React. Em outras palavras:

- React: biblioteca para construir interfaces;
- Vite: ferramenta para iniciar e executar o projeto localmente.

No terminal, dentro da pasta de projetos, os alunos executam:

```
None
npm create vite@latest todo-pro
cd todo-pro
npm install
npm run dev
```

6.2 Evolução do React ao longo dos anos

O React não nasceu como uma solução fechada e estática. Ele passou por diferentes fases de evolução, e cada mudança reflete a maturidade da biblioteca e as necessidades do mercado.

React 16 e 17: base sólida e crescimento da comunidade

As primeiras versões do React começaram a consolidar o modelo de componentes e a ideia de renderização declarativa. Em um momento em que os frameworks JavaScript estavam se tornando muito complexos, o React mostrou que era possível criar interfaces mais previsíveis e reutilizáveis.

A comunidade começou a crescer rapidamente, e o React passou a ser usado em projetos grandes, de startups a empresas de grande porte. A maior mudança dessa fase foi a popularização do componente como unidade de organização e a consolidação do JSX como padrão na criação das interfaces.

React 18: atualização de performance e concorrência

A versão 18 foi um marco importante. Ela trouxe melhorias significativas em desempenho e em como o React lida com renderização e atualização de interface, com foco em recursos como:

- renderização concorrente;
- melhor uso de CPU e memória;
- suporte a atualizações mais rápidas e responsivas;
- melhorias em ferramentas e compatibilidade com ambientes modernos.

Essa evolução foi importante porque aplicações web passaram a exigir interfaces mais responsivas, principalmente com dados em tempo real, listas grandes e experiências de usuário mais fluidas.

React 19 e o futuro: modernização e simplicidade

Com as versões mais recentes, o foco tem sido a simplificação do desenvolvimento e a integração com novas formas de escrever componentes, utilizando melhor suporte a integrações reativas, renderização mais eficiente e APIs mais modernas.

O React também foi se aproximando de um modelo mais declarativo, com evoluções em APIs e abstrações que facilitam a criação de interfaces complexas sem deixar o código excessivamente verboso. A ideia central permanece a mesma: descrever a interface a partir do estado e permitir que o React cuide da atualização da tela.

Isso demonstra que o React não ficou parado. Ele evoluiu para atender:

- aplicações maiores;
- melhor desempenho;
- mais previsibilidade;
- melhor experiência de desenvolvimento;
- componentes mais reutilizáveis e fáceis de manter.

Uma maneira simples de entender essa evolução é pensar assim:

- no começo, o React era uma resposta à complexidade da manipulação do DOM;
- depois, ele se tornou a base de aplicações frontend robustas;
- hoje, ele continua evoluindo para lidar com demandas modernas de performance, arquitetura e produtividade.

6.3 Primeiro contato com a estrutura do projeto

Abra a pasta do projeto com o VS Code e mostre os itens principais:

Item	Função
`package.json`	scripts e dependências
`package-lock.json`	versões exatas instaladas
`node_modules/`	bibliotecas baixadas

`public/`	arquivos estáticos
`src/`	código principal
`src/main.jsx`	entrada da aplicação
`src/App.jsx`	componente principal
`src/index.css`	estilos globais
`.gitignore`	arquivos excluídos do Git

Explique que a relação é esta:

```
None
index.html -> main.jsx -> App.jsx -> navegador
```

Atividade de sala: exploração orientada

Os alunos devem responder em um arquivo `estrutura-projeto.md`:

1. Qual arquivo contém os scripts do npm?
2. Por que `node_modules` não deve ir para o Git?
3. Onde fica o código principal da aplicação?
4. Qual é o papel de `main.jsx`?
5. Qual comando gera a versão final da aplicação?

---

7. Bloco 4 — JSX e componentes

7.1 O que é JSX?

JSX é uma sintaxe que permite escrever algo parecido com HTML dentro do JavaScript.

Exemplo:

```
None
function Welcome() {
```

```
  const name = "Equipe To-Do Pro";

  return (
    <section>
      <h1>Bem-vindo, {name}</h1>
      <p>Organize suas tarefas.</p>
    </section>
  );
}
```

Explicação do professor

- JSX não é HTML puro;
- ele é convertido em código JavaScript pelo processo do build;
- expressões JavaScript ficam entre chaves `{}`;
- a função retorna um único elemento raiz;
- nomes de componentes começam com letra maiúscula.

7.2 Substituindo o conteúdo inicial

Abrir `src/App.jsx` e substituir o conteúdo pelo seguinte:

None

```
function App() {
  return (
    <main>
      <header>
        <h1>To-Do Pro</h1>
        <p>Organize suas tarefas em um só lugar.</p>
      </header>

      <section>
        <h2>Minhas tarefas</h2>
        <p>Nenhuma tarefa cadastrada ainda.</p>
        <button type="button">Adicionar tarefa</button>
      </section>
    </main>
  );
}
```

```
export default App;
```

Atividade de sala: personalização do JSX

Alterem a tela para incluir:

1. o nome do aluno ou da equipe;
2. uma frase explicando o problema resolvido;
3. o texto `0 tarefas pendentes`;
4. três tarefas de exemplo;
5. um botão chamado `Nova tarefa`.

Diga que ainda não é necessário deixar a interface interativa. O objetivo agora é apenas provar que o JSX funciona e a interface aparece corretamente.

---

8. Bloco 5 — Estado local com `useState`

8.1 O conceito de estado

Uma tela real precisa responder a eventos do usuário. Isso inclui:

- digitar no campo;
- marcar uma tarefa como concluída;
- remover um item;
- alterar filtros.

Esses dados são chamados de estado.

Mostre o exemplo:

```
None
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
```

```
  return (
    <button type="button" onClick={() => setCount(count + 1)}>
      Cliques: {count}
    </button>
  );
}
```

Explicação passo a passo

- `count` é o valor atual;
- `setCount` é a função que atualiza o valor;
- `useState(0)` define o valor inicial;
- quando o botão é clicado, o estado muda e o React reexecuta o componente.

8.2 Por que não usar uma variável comum?

O erro clássico:

None
```
function WrongCounter() {
  let count = 0;

  function increment() {
    count += 1;
    console.log(count);
  }

  return <button onClick={increment}>Cliques: {count}</button>;
}
```

Esse código altera uma variável local, mas não comunica ao React que a interface precisou mudar. O texto não atualiza corretamente na tela.

8.3 Atualização funcional do estado

Quando o novo valor depende do anterior, o ideal é usar a forma funcional:

None
```
setCount((currentCount) => currentCount + 1);
```

Use o exemplo de somar 3 vezes em sequência:

```
None
function Counter() {
  const [count, setCount] = useState(0);

  function incrementThreeTimes() {
    setCount((current) => current + 1);
    setCount((current) => current + 1);
    setCount((current) => current + 1);
  }

  return (
    <>
      <p>Valor: {count}</p>
      <button type="button" onClick={incrementThreeTimes}>Somar
3</button>
    </>
  );
}
```

8.4 Imutabilidade de objetos e arrays

Em React o estado deve ser tratado como imutável. Isso é crucial para o funcionamento correto da interface.

O que não fazer:

```
None
// Evite
tasks.push(newTask);
setTasks(tasks);
```

E o que fazer:

```
None
setTasks((currentTasks) => [...currentTasks, newTask]);
```

`map`, `filter` e spread:

```
None
setTasks((currentTasks) =>
  currentTasks.map((task) =>
    task.id === taskId ? { ...task, completed: !task.completed }
: task
  )
);
```

O objetivo é criar uma nova referência em vez de modificar a existente.

8.5 Estado derivado

Nem tudo precisa ser salvo em estado separado. Se um valor pode ser calculado a partir de outro, não é necessário criar um segundo estado.

```
None
const completedCount = tasks.filter((task) =>
task.completed).length;
```

Diga que isso evita inconsistência entre dados.

8.6 Regras dos hooks

- use hooks no nível superior do componente;
- não use `useState` dentro de `if`, `for` ou funções internas;
- mantenha o estado mínimo necessário;
- passe dados e callbacks por `props`.

Exemplo de aplicação em sala

Crie um componente de exemplo com tarefas:

```
None
import { useState } from "react";

function TaskSummary() {
  const [tasks, setTasks] = useState([
    { id: 1, title: "Estudar React", completed: false },
    { id: 2, title: "Criar projeto Vite", completed: true },
  ]);
```

```
  const completedCount = tasks.filter((task) =>
task.completed).length;

  function toggleTask(taskId) {
    setTasks((currentTasks) =>
      currentTasks.map((task) =>
        task.id === taskId ? { ...task, completed:
!task.completed } : task
      )
    );
  }

  return (
    <section>
      <p>Concluídas: {completedCount}</p>
      <ul>
        {tasks.map((task) => (
          <li key={task.id}>
            <span>{task.title}</span>
            <button type="button" onClick={() =>
toggleTask(task.id)}>
              {task.completed ? "Reabrir" : "Concluir"}
            </button>
          </li>
        ))}
      </ul>
    </section>
  );
}
```

Atividade de sala: contador de tarefas

Criar um componente `TaskSummary` com:

1. pelo menos 3 tarefas;
2. total de tarefas;
3. total de concluídas;
4. botão para alternar status;
5. `key` com identificador;
6. sem uso de `push`, `splice` ou mutação direta.

Em seguida, faça a discussão:

- por que `setTasks((currentTasks) => ...)` é mais seguro?
- por que `completedCount` não precisa ser outro estado?
- o que acontece no DOM quando conclui-se uma tarefa?

---

9. Bloco 6 — Git e documentação inicial

9.1 Versionando o projeto

Git ajuda a registrar cada etapa do desenvolvimento.

No terminal, dentro da pasta do projeto:

```
None
git init
git status
git add .
git commit -m "feat: cria base inicial do To-Do Pro"
```

Mostre o resultado do `git status` e explique o que está sendo salvo.

9.2 README

Explique que o README serve como documentação do projeto e deve responder:

- o que é o projeto;
- quais ferramentas são necessárias;
- como instalar;
- como executar;
- como fazer build final.

Exemplo de README:

```
None
# To-Do Pro

Aplicação React para organização de tarefas.
```

```
## Pré-requisitos
- Node.js LTS
- npm

## Instalação
```bash
npm install
```

Execução

None
```
npm run dev
```

None
```
### Atividade de sala

Peça aos alunos para criar ou editar o `README.md` no projeto.
Após isso, verifique se o projeto pode ser executado pelos
colegas sem ajuda extra.

---

## 10. Desafio final da semana: To-Do Pro v1

### 10.1 Apresentação do contexto

Explique que a empresa não quer apenas uma interface bonita, e
sim uma solução funcional e que comporte um fluxo real de uso.
Neste primeiro momento, os dados podem ficar em memória. A ideia
é validar a lógica e a experiência do usuário.

### 10.2 Modelo mínimo da tarefa

Apresente o objeto:
```
```

```js
{
  id: 1,
  title: "Preparar apresentação",
  description: "Revisar os tópicos da reunião semanal",
  priority: "high",
  completed: false,
  createdAt: "2026-09-10"
}
```

10.3 Requisitos funcionais

Apresente os requisitos em bloco, mas sem perder o foco em sequência de desenvolvimento.

Cadastro

- formulário com título obrigatório;
- descrição opcional;
- prioridade com `Baixa`, `Média` e `Alta`;
- validação do título;
- limpar o formulário após adicionar;
- atualizar contadores após cadastro.

Gerenciamento

- concluir e reabrir tarefa;
- remover tarefa;
- filtrar por `Todas`, `Pendentes` e `Concluídas`;
- mostrar totais derivados;
- mostrar mensagens de estado vazio.

Componentização e acessibilidade

- criar `TaskForm`, `TaskSummary`, `TaskFilters`, `TaskList` e `TaskItem`;
- usar `label` corretamente;
- utilizar `main`, `header`, `section`, `form`, `ul`, `li` com semântica adequada;
- manter a interface legível em telas estreitas.

10.4 Restrições técnicas

O que é permitido e o que deve ser evitado:

- usar React, JSX, `useState` e Vite;

- manter o estado principal no componente pai;
- atualizar arrays sem mutação direta;
- não usar backend nesta semana;
- não manipular o DOM diretamente com `querySelector`;
- não versionar `node_modules`, senha ou token.

10.5 Roteiro de implementação

Seguir esta sequência:

1. Definir o estado inicial com três tarefas.
2. Criar o formulário de cadastro e validar o título.
3. Separar os componentes e passar props.
4. Implementar concluir e remover.
5. Aplicar filtros.
6. Derivar contadores e lista visível.
7. Testar manualmente.
8. Registrar os testes em `TESTES.md`.

10.6 Casos de teste manuais

Apresente a tabela:

Caso	Ação	Resultado esperado
1	Enviar formulário vazio	Mensagem de validação
2	Título com dois caracteres	Mensagem de validação
3	Cadastrar tarefa válida	Tarefa aparece na lista
4	Concluir e reabrir	Status visual e contadores atualizados
5	Aplicar filtro sem resultados	Mensagem de lista vazia
6	Remover última tarefa	Estado vazio adequado
7	Redimensionar a janela	Layout continua legível

10.7 Checklist de entrega

Leitura em aula e revisão final:

`npm install`, `npm run dev` e `npm run build` sem erro
README com instruções de instalação e execução
formulário validado
concluir, reabrir, remover e filtrar funcionando
contadores derivados de `tasks`
estados vazio e filtro sem resultados distintos
componentes com responsabilidades claras
`TESTES.md` registrado
pelo menos dois commits bem descritos
sem `node_modules` ou dados sensíveis no Git

---

11. Autoavaliação em sala

Termine a aula com perguntas reflexivas:

1. O que você aprendeu sobre frontend, backend e fullstack?
2. Qual a principal diferença entre Angular e React?
3. O que é JSX e para que serve?
4. Por que o estado precisa ser tratado com imutabilidade?
5. Qual foi a parte mais difícil até agora?

A autoavaliação pode ser feita em tabela no caderno ou em formulário:

Afirmação	Ainda não	Com ajuda	Sozinho
Consigo verificar Node.js, npm e Git.			
Consigo criar projeto React com Vite.			
Consigo iniciar e encerrar o servidor local.			
Consigo escrever um componente com JSX.			
Consigo registrar alterações com Git.			
Consigo explicar o funcionamento do estado.			

---

12. Glossário rápido para sala de aula

- API: interface usada para comunicação entre sistemas.
- Componente: unidade reutilizável de interface.
- Frontend: camada visual e interativa da aplicação.
- JSX: combinação de JavaScript e marcação.
- npm: gerenciador de pacotes do Node.js.
- React: biblioteca para criação de interfaces.
- SPA: aplicação que atualiza partes da interface sem recarregar a página inteira.
- Vite: ferramenta de desenvolvimento frontend.

---

13. Referências sugeridas

- React. Learn React. Disponível em: https://react.dev/learn
- Vite. Getting Started. Disponível em: https://vite.dev/guide/
- Node.js. Documentação oficial. Disponível em: https://nodejs.org/docs/latest/api/
- Git. Documentação oficial. Disponível em: https://git-scm.com/doc
- MDN Web Docs. JavaScript. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript

---

14. Objetivo de aprendizagem final da semana

Ao concluir esta semana, a turma deve ter entendido a lógica básica do React, montado o ambiente de desenvolvimento, criado um projeto com Vite, compreendido JSX e estado e iniciado a base da aplicação To-Do Pro. Esse é o ponto de partida para as próximas aulas, nas quais o projeto será ampliado com novas funcionalidades, organização por componentes e lógica mais complexa de dados e interface.